

Student: Jakub Kwaśniewski

Numer albumu 32430

Sprawozdanie: Listy

Metody Programowania

Uczelnia: Politechnika Morska w Szczecinie

Data: 11.05.2026

Wstęp do zadania:

Celem ćwiczenia było zapoznanie się z implementacją i obsługą dynamicznych struktur danych, list jednokierunkowych, dwukierunkowych oraz cyklicznych. Listy te stanowią fundament efektywnego zarządzania pamięcią w programowaniu, pozwalając na elastyczne dodawanie i usuwanie elementów bez konieczności rezerwowania ciągłego obszaru pamięci.

- 1 Przepisz powyższe fragmenty kodu. W funkcji main() utwórz nową listę jednokierunkową i wykonaj testy każdej funkcji [pushFront, pushBack, popFront i printList]. Następnie opisz działanie struktury Node oraz każdej funkcji linijka po linijce.

```
int main()
{
    Node* mojaLista = nullptr;
    pushFront(mojaLista, 10);
    pushFront(mojaLista, 20);
    printList(mojaLista);

    pushBack(mojaLista, 30);
    pushBack(mojaLista, 40);
    printList(mojaLista);

    popFront(mojaLista);
    printList(mojaLista);
}
```

```
20 -> 10 -> NULL
20 -> 10 -> 30 -> 40 -> NULL
10 -> 30 -> 40 -> NULL
```

```

struct Node
{
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr){}
};

```

struct Node{}; - Definicja struktury reprezentującej pojedynczy element listy.

int data; - Pole przechowujące dane

Node* next; - Wskaźnik na kolejny element typu Node w liście

Node(int val) : data(val), next(nullptr) {} - Konstruktor, który ustawia wartość data na przekazaną w argumencie

```

void pushFront(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head;
    head = newNode;
}

```

Node* newNode = new Node(val); - Tworzy nowy węzeł w pamięci

newNode->next = head; - Ustawia wskaźnik nowego węzła tak, aby wskazywał na dotychczasowy pierwszy element listy.

head = newNode; - Przypisuje adres nowego węzła jako nowy początek listy.

```

void pushBack(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (!head) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

```

Node* newNode = new Node(val); - Tworzy nowy węzeł

if (!head) {

head = newNode;

return;

} - Jeśli lista jest pusta, nowy węzeł staje się jej głową.

Node* temp = head; - Tworzy tymczasowy wskaźnik do nawigacji po liście.

while (temp->next) temp = temp->next; - Przesuwa wskaźnik temp aż do momentu gdy znajdzie ostatni element

temp->next = newNode; - Podłącza nowy węzeł na samym końcu

```
void popFront(Node*& head) {  
    if (!head) return;  
    Node* temp = head;  
    head = head->next;  
    delete temp;  
}
```

if (!head) return; - Jeśli lista jest pusta, nic nie rób.

Node* temp = head; - Zapamiętuje adres aktualnego pierwszego elementu, aby móc go później usunąć.

head = head->next; - Przesuwa początek listy na drugi element.

delete temp; - Zwalnia pamięć zajmowaną przez stary pierwszy element.

```
void printList(Node* head) {  
    Node* temp = head;  
    while (temp) {  
        cout << temp->data << " -> ";  
        temp = temp->next;  
    }  
    cout << "NULL" << endl;  
}
```

Node* temp = head; - Zaczyna przeglądanie od początku listy.

while (temp) {

cout << temp->data << " -> ";

temp = temp->next;

} - Pętla wykonuje się, dopóki wskaźnik temp nie pokaże na nullptr.

cout << "NULL" << endl; - koniec listy;

- 2 Napisz pętlę, która wygeneruje listę jednokierunkową o liczbie węzłów podanej przez użytkownika z pseudolosowymi liczbami jako wartościami węzłów.

```
Node* mojaLista2 = nullptr;
int n;
cout << "Podaj liczbę węzłów do wygenerowania: ";
cin >> n;
for (int i = 0; i < n; i++) {
    int losowaWartosc = rand() % 100;
    pushBack(mojaLista2, losowaWartosc);
}
printList(mojaLista2);
```

```
Podaj liczbę węzłów do wygenerowania: 2
41 -> 67 -> NULL
```

- 3 Napisz funkcję, która zwróci liczbę węzłów w liście jednokierunkowej.

```
int countNodes(Node* head) {
    int count = 0;
    Node* temp = head;
    while (temp != nullptr) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

```
int liczbaWezlow = countNodes(mojaLista2);
cout << "Liczba węzłów w liście: " << liczbaWezlow << endl;
```

```
Liczba węzłów w liście: 1
```

- 4 Porównaj, czy liczba podana przez użytkownika [punkt 2.] dotycząca liczby węzłów w liście zgadza się z wynikiem funkcji z punktu 3.

```
if (liczbaWezlow == n) {
    cout << "Liczba węzłów zgadza się" << " " << n << endl;
}
else {
    cout << "Liczba węzłów jest inna" << endl;
}
```

```
Liczba węzłów w liście: 1
Liczba węzłów zgadza się 1
```

5 Napisz funkcję o nazwie popBack, która usunie węzeł z końca listy.

```
void popBack(Node*& head) {
    if (head == nullptr) return;
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;
    }
    Node* temp = head;
    while (temp->next->next != nullptr) {
        temp = temp->next;
    }
    delete temp->next;
    temp->next = nullptr;
}
```

```
popBack(mojaLista);
printList(mojaLista);
```

```
20 -> 10 -> NULL
20 -> 10 -> 30 -> 40 -> NULL
10 -> 30 -> 40 -> NULL
10 -> 30 -> NULL
```

6 Napisz funkcję, która odszuka w liście jednokierunkowej węzeł o wartości podanej przez użytkownika oraz zwróci adres tego węzła.

```
Node* findNode(Node* head, int value) {
    Node* temp = head;
    while (temp != nullptr) {
        if (temp->data == value) {
            return temp;
        }
        temp = temp->next;
    }
    return nullptr;
}
```

```
int szukana;
cout << "Podaj wartość do odszukania: ";
cin >> szukana;

Node* wynik = findNode(mojaLista2, szukana);

if (wynik != nullptr) {
    cout << "Znaleziono węzeł o wartości " << szukana << " pod adresem: " << wynik << endl;
}
else {
    cout << "Wartość " << szukana << " nie istnieje w liście" << endl;
}
```

```
Podaj wartość do odszukania: 2
Wartość 2 nie istnieje w liście
```

7 Przepisz powyższy kod dla list dwukierunkowych. Uruchom. Sprawdź działanie.

```
DNode* head = nullptr;
pushFront(head, 10);
pushFront(head, 20);
pushFront(head, 30);

cout << "Zawartość listy: ";
DNode* temp = head;

while (temp != nullptr) {
    cout << temp->data;
    if (temp->next != nullptr) cout << " <-> ";
    temp = temp->next;
}
cout << " -> NULL" << endl;
```

```
Zawartość listy: 30 <-> 20 <-> 10 -> NULL
```

8 Napisz pętlę, która wygeneruje listę dwukierunkową o liczbie węzłów podanej przez użytkownika z pseudolosowymi liczbami jako wartościami węzłów.

```
int n;
cout << "Ile losowych elementów dodać do listy ";
cin >> n;

for (int i = 0; i < n; i++) {
    pushFront(head, rand() % 100);
}
```

```
Ile losowych elementów dodać do listy 1
Zawartość listy: 41 <-> 30 <-> 20 <-> 10 -> NULL
```

9 Zaimplementuj mechanizm listy cyklicznej – minimum to struktura węzła, możliwość dodania węzła i usunięcia węzła.

```
struct Node
{
    int data;
    Node* next;
    Node* tail;
    Node(int val) : data(val), next(tail), tail(next) {}
};
```



```

void pushFront(Node*& head, int val) {
    Node* newNode = new Node(val);
    Node* temp = head->next;
    newNode->next = head;
    newNode->tail = temp;
    head = newNode;
}

```

```

void popFront(Node*& head) {
    if (!head) return;
    Node* temp = head;
    head->tail = head->next;
    head = head->next;
    delete temp;
}

```

```

void printList(Node* head) {
    if (!head) return;
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

```

```

int main()
{
    setlocale(LC_CTYPE, "Polish");

    Node* head = nullptr;
    pushFront(head, 10);
    pushFront(head, 20);
    pushFront(head, 30);
    printList(head);
    popFront(head);
    printList(head);

    return 0;
}

```

```

30 -> 20 -> 10 -> NULL
20 -> 10 -> NULL

```

Wnioski:

dynamiczne struktury danych w postaci list jednokierunkowych, dwukierunkowych oraz cyklicznych. Zaobserwowano, że listy pozwalają na elastyczne zarządzanie pamięcią, ponieważ ich rozmiar może być zmieniany w trakcie działania programu. Każdy węzeł przechowuje dane oraz wskaźniki do sąsiednich elementów co umożliwia nawigację po strukturze. Wybór konkretnego typu listy zależy od potrzeb, listy jednokierunkowe są oszczędniejsze pamięciowo, dwukierunkowe ułatwiają nawigację w obu strony, a cykliczne zapewniają stały dostęp do danych w pętli.